Добавь асинхронный цикл для агентов

Добавим **асинхронный цикл**: агенты работают параллельно через `asyncio`, а координатор собирает их решения. Это приближает модель к реальному рою: не «по очереди», а **одновременно**.

1. Обновим `BaseAgent`

Сделаем методы `evaluate` и `act` асинхронными.

```
# src/les/agents/base_agent.py
from abc import ABC, abstractmethod
from typing import Dict, Any

class BaseAgent(ABC):
    def __init__(self, name: str, threshold: float = 0.3):
        self.name = name
        self.threshold = threshold

    @abstractmethod
    async def evaluate(self, state: Dict[str, float]) -> Dict[str, Any]:
        pass

    @abstractmethod
    async def act(self, state: Dict[str, float]) -> Dict[str, Any]:
        pass
```

2. Обновим агентов

Пример для `SurvivalAgent`:

```
# src/les/agents/survival_agent.py
from .base_agent import BaseAgent

class SurvivalAgent(BaseAgent):
    async def evaluate(self, state):
        S = state['S']
        return {
            'risk': S < self.threshold,
            'value': S
        }

    async def act(self, state):
        eval_result = await self.evaluate(state)
        if eval_result['risk']:
```

```
            return {
                'action': 'escalate',
                'reason': 'S below threshold',
                'agent': self.name
            }
        return {
            'action': 'continue',
            'agent': self.name
        }
```

Аналогично обновляются все остальные агенты.

3. Обновим `SwarmManager`

```
# src/les/swarm/swarm_manager.py
import asyncio
from typing import Dict, List, Any
from les.agents import (
    SurvivalAgent,
    OthersAgent,
    MeaningAgent,
    ProcedureAgent,
    AdaptationAgent,
    DiversityAgent
)

class SwarmManager:
    def __init__(self):
        self.agents = [
            SurvivalAgent('survival'),
            OthersAgent('others'),
            MeaningAgent('meaning'),
            ProcedureAgent('procedure'),
            AdaptationAgent('adaptation'),
            DiversityAgent('diversity'),
        ]
        self.log = []

    async def run(self, state: Dict[str, float]) -> Dict[str, Any]:
        tasks = [agent.act(state) for agent in self.agents]
        results = await asyncio.gather(*tasks)

        conflicts = [r for r in results if r['action'] in ('escalate', 'block')]
        self.log.extend(results)

        decision = self.decide(results, conflicts)
        return {
            'state': state,
            'results': results,
            'conflicts': conflicts,
            'decision': decision
        }

    def decide(self, results, conflicts):
        if any(c['action'] == 'block' for c in conflicts):
```

```
            return 'STOP'
        if conflicts:
            return 'ESCALATE'
        return 'CONTINUE'
```

4. Обновим `main.py`

```
import asyncio
from les.swarm import SwarmManager

async def main():
    state = {
        'S': 0.2,
        'O': 0.8,
        'M': 0.8,
        'P': 0.8,
        'A': 0.8,
        'D': 0.8
    }

    manager = SwarmManager()
    result = await manager.run(state)

    print("Решение координатора:", result['decision'])
    print("Конфликты:", result['conflicts'])

if __name__ == "__main__":
    asyncio.run(main())
```

5. Что изменилось

- `evaluate` и `act` стали `async`.
- `SwarmManager.run` стал `async`.
- Агенты запускаются **параллельно** через `asyncio.gather`.
- `main` стал асинхронным и запускается через `asyncio.run`.

6. Почему это важно

Асинхронность позволяет:

- агентам работать **одновременно**;
- не блокировать друг друга;
- масштабировать рой;
- добавлять задержки, сетевые вызовы, ИИ-модели.

Это стандартный паттерн: `asyncio.create_task()` + `asyncio.gather()` для параллельного выполнения задач.[1] [2] [3] [4]

7. Финальная формулировка

> **Рой — это не «очередь», а «оркестр»: все играют одновременно, но в общем ритме.**

Асинхронный цикл делает модель **живой**: агенты не ждут друг друга, а реагируют параллельно. [2] [5] [6] [1]

⁂

1. https://docs.python.org/uk/3/library/asyncio-task.html
2. https://docs.python.org/3/howto/a-conceptual-overview-of-asyncio.html
3. https://docs.python.org/3/library/asyncio-task.html?highlight=gather
4. https://docs.python.org/3/library/asyncio-task.html?highlight=get_coro
5. https://developers.llamaindex.ai/python/llamaagents/workflows/async_workflows/
6. https://github.com/academy-agents/academy/blob/main/academy/agent.py
7. https://langchain-doc.readthedocs.io/en/latest/_sources/modules/agents/examples/async_agent.ipynb
8. https://volttron.readthedocs.io/en/9.0.4/developing-volttron/developing-agents/using-asyncio-in-agents.html
9. https://volttron.readthedocs.io/en/releases-9.x/developing-volttron/developing-agents/using-asyncio-in-agents.html
10. https://github.com/avkghost/agent-loop/blob/master/EXAMPLES.md
11. https://github.com/MervinPraison/PraisonAI/blob/main/examples/python/general/async_example.py
12. https://github.com/wshobson/agents/blob/4bde3168b43c0655c4d00ea2726e631719908a6d/plugins/python-development/skills/async-python-patterns/references/details.md
13. https://docs.python.org/3/library/asyncio-dev.html
14. https://docs.python.org/3/library/asyncio-eventloop.html
15. https://docs.python.org/3/library/asyncio-runner.html